Spurðu Hagstofuna: the whole system

How a question becomes a cited number, what is built offline, how it is measured, and what is prepared for LUMI. Code state 29 October 2026 (branch claude/wonderful-heisenberg-s3mbqr).

LLM decides (gemma4:26b today) · Our code · Miðeind tool (Icelandic) · External service / user · Stored data / cache · Planned, not built yet

① Offline

Built once, before any question is asked

EXTERNAL **PX-Web API** — px.hagstofa.is, Statistics Iceland's open database

→ CODE **Crawl the catalog** — walks every folder, 1 request per second, ~1 hour — `scripts/crawl_catalog.py`

→ DATA **Table catalog: 2,102 tables** — titles IS + EN, folders, variables, every value code and text, path, last updated — `data/catalog.jsonl`

→ CODE **raw index** — TF-IDF over character 3–5-grams of titles, variables, first 40 values, English

→ MIÐEIND **lemma index** — TF-IDF over BÍN base forms of the same text (town names left out)

→ DATA **meaning vectors** — bge-m3: 2,102 × 1,024 numbers — `data/emb_bge-m3_*.npy`

→ CODE **Places index** — municipality names + aliases from shared 4-digit codes: Kópavogsbær = Kópavogur

↓ these four indexes are loaded into memory when the app starts

② Question in

Normalise the question and split it if it needs two tables

USER **A question** — Icelandic or English, typos allowed

→ CODE **Chat UI** — Gradio — `app.py`

PLANNED **FastAPI server** — POST /spyrja_px_api, same fields as the Track 2 starter — `main.py`

→ CODE **English?** — 2+ English function words and no Icelandic letters (right on all 810 questions it was checked on) — `is_english()`

Icelandic → MIÐEIND **Fix spelling** — GreynirCorrect: typos, accents, capitals. kopavogi → Kópavogi — `correct_question() · cache data/corrections.sqlite`

English → **skip correction** — (it would turn English into Icelandic)

→ LLM **Split if it needs two tables** — "population AND inflation" → 2 self-contained questions; otherwise kept as asked — `decompose() · max 2 parts`

↓ each part (1 or 2) goes through ③ and ④ on its own

③ Search

Once per part. Finds the 8 most likely tables among 2,102 in a few milliseconds

CODE **Add official terms** — SYNONYMS: verðbólga → vísitala neysluverðs; búa → mannfjöldi — `expand_query()`

→ CODE **raw score** — shared character n-grams; accents folded on both sides (haskolanams = háskólanáms)

→ MIÐEIND **lemma score** — shared base forms (Kópavogi → kópavogur); × 1.5 for tables containing a town named in the question

→ CODE **meaning score** — question → bge-m3 vector (~0.5 s on CPU, cached), then one dot product per table: O(N·d) ≈ 1 ms — `cache data/question_vectors.sqlite`

→ CODE **Archive × 0.6** — older tables (eldra efni, Sögulegar, census) rank lower

→ CODE **Rank fusion (RRF)** — score = Σ 1 / (60 + rank) over the three; × 1.2 for tables with the named town

→ CHECK **Checker on top 20** — tables lacking the place, country or year asked about move down (BÍN place categories) — `checker.py`

→ RESULT **Top 8 tables** — right one in here for 86% of dev questions

↓

④ Numbers

Once per part. The model chooses; code checks, fetches and computes

LLM **Pick the table** — one of the 8, from titles and variable names — `pick_table()`

none fits → **part noted as "no data found"**

→ CODE **List the options** — each variable's codes and texts; long lists trimmed to 60 — `describe_variables()`

→ LLM **Choose values** — codes as JSON, e.g. {"Sveitarfélag": ["0000"], "Ár": ["2025","2026"]}; every value a comparison needs

→ CODE **Validate** — drop codes that don't exist (no invented values); fill required variables with the latest period or "Alls" — `validate_selections()`

⇄ EXTERNAL **Fetch live numbers** — POST to PX-Web, json-stat2; 0.4 s pacing, retries on busy, timeout and network errors — `px.query() → flatten() → rows`

→ 2+ rows → LLM **Name ONE operation** — difference · % change · share · ratio · sum · average, and which rows

→ CODE **Compute** — checks rows and division by zero: 139.804 − 138.772 = 1.032 — `calculate()`

1 row → **no calculation needed**

↓ all parts meet again here

⑤ Answer

One answer over all parts, with a source per table

LLM **Write the answer** — from the fetched rows only (≤ 200 per part); computed results handed over "as is"; names each table and period; says plainly what wasn't found — `write_answer()`

→ CODE **Add the citation** — table id, title, PX-Web link and last-updated date, one line per table used. Code, so it can't be hallucinated

→ USER **Answer + sources**

◀ PLANNED **Next additions** — unit from the data passed to the answer · values used shown in the UI · stated assumptions ("I assumed 2026, both sexes") · response time · answer language decided by code · forced table choice

LLM endpoint

One setting, any OpenAI-compatible server

EXTERNAL **Ollama or vLLM** — gpu-talva today (4 GB GPU + CPU) · DGX Spark or LUMI at the event
LLM_MODEL = gemma4:26b (Apache-2.0, MoE, ~4B active) · LLM_REASONING = none · JSON mode · 30-min timeout — `hagstofa/llm.py · LLM_BASE_URL`

LLM CALLS **Up to 5 per question** — split · pick table · choose values · operation · write answer

⑥ Measurement

Decide on dev, confirm once on test

DATA **Question sets** — hand 36 · dev 270 · test 627 (frozen) · municipalities 24 · real 11 · misspelled 540 · e2e 36 (24 easy, 7 calculation, 5 two-table)

→ CODE **run_eval.py** — top 1 / 3 / 8, right folder, --pick, --e2e

CODE **compare_models.py** — pick · oracle (right table given) · full: number, values, seconds

⑦ Prepared for LUMI

Fine-tuning the search model (a Track 3-style extension)

LLM **Write training questions** — 5 per table, only for the ~950 current tables in no eval set, so dev stays a fair test — `scripts/finetune_questions.py`

→ CODE **Contrastive training** — question and its table pulled together, the other tables in the batch pushed apart; bge-m3 vs Miðeind's IceBERT — `scripts/finetune_embed.py`

→ DATA **New search model** — switched in with EMBED_MODEL=models/...; measured on dev against bge-m3 (IceBERT untrained: 7% top 1)

→ CODE **Replaces the meaning vectors in ① and ③** — only if it beats bge-m3 on dev

Where the time goes

The search itself takes milliseconds. Almost all of an answer's time is the up-to-five LLM calls, so faster hardware (DGX Spark) and fewer calls matter far more than a faster index.

Why it is built this way

The model only chooses (table, values, operation, wording). Code validates the codes, does the arithmetic and adds the source, so numbers and citations can't be invented.

Measured so far

Right table in the top 8: 86% (embedding-only search: 65%). gemma4:26b given the right table: right number 24 of 24. Calculations and two-table questions: built and unit-tested, not yet measured with a real model.